

1.1 PROJECT TITLE

Network Traffic Analysis and Packet Investigation Using Wireshark

1.2 PROJECT OVERVIEW

- This project focuses on analyzing and investigating network traffic using Wireshark, a widely used network protocol analyzer.
- A controlled cybersecurity lab environment was created using Kali Linux and Metaspolitable 2 virtual machines. Different types of network traffic were generated and captured, including ARP, ICMP, TCP, HTTP, DNS, and Nmap scanning traffic.
- The captured packets were analysed to understand how devices communicate over a network and how network analysts can identify different protocols, connections, and suspicious activities.

1.3 OBJECTIVES

- To understand packet capture and network traffic analysis.
- To learn how Wireshark captures network packets.
- To analyze ARP requests and responses.
- To analyze ICMP traffic generated using ping.
- To understand TCP connection establishment.
- To identify TCP SYN, SYN-ACK and ACK packets.
- To capture and analyze HTTP traffic.
- To use Wireshark filters for protocol-specific investigation.
- To analyze conversations and TCP streams.
- To understand how packet analysis can help in cybersecurity investigations.

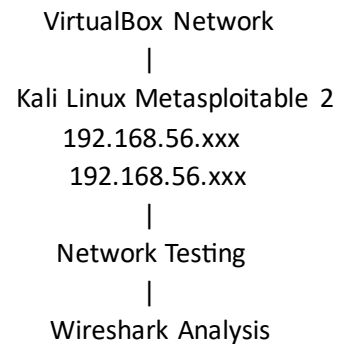
1.4 LAB ENVIRONMENT & TOOLS USED

System Info & Tool	Purpose
--------------------	---------

Hardware (Laptop/PC), RAM: 16GB	Generate Windows Security Logs
Virtualization (Oracle VirtualBox)	Virtual machine environment
Kali Linux	Security testing & traffic-generated machine
Metaspolitable 2	Target Machine
Wireshark	Packet capture and network analysis
Nmap	Network Scanning
Ping	ICMP traffic generation
Browser	HTTP packet generation
tcpdump	Packet capture when required

1.5 NETWORK ARCHITECTURE

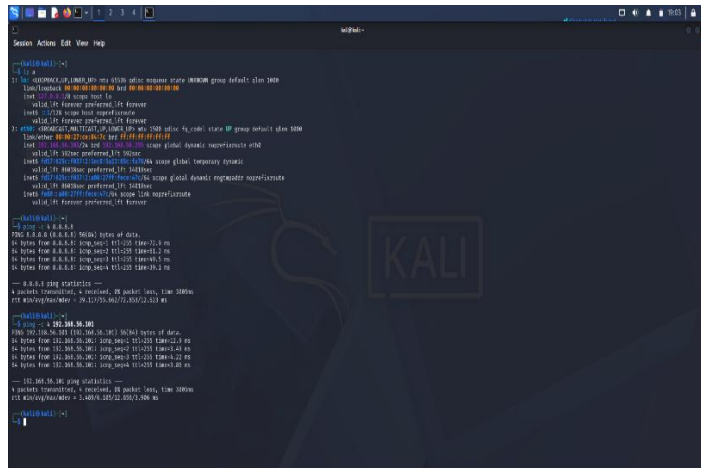
- The project used an isolated virtual network between Kali Linux and Metaspolitable 2.



1.6 INITIAL NETWORK CONFIGURATION

- The IP addresses of the virtual machines were checked before starting packet analysis.

- 1) Kali Linux
Ip addr → 192.168.56.xxx
- 1) Metasploitable 2
ifconfig → 192.168.56.xxx

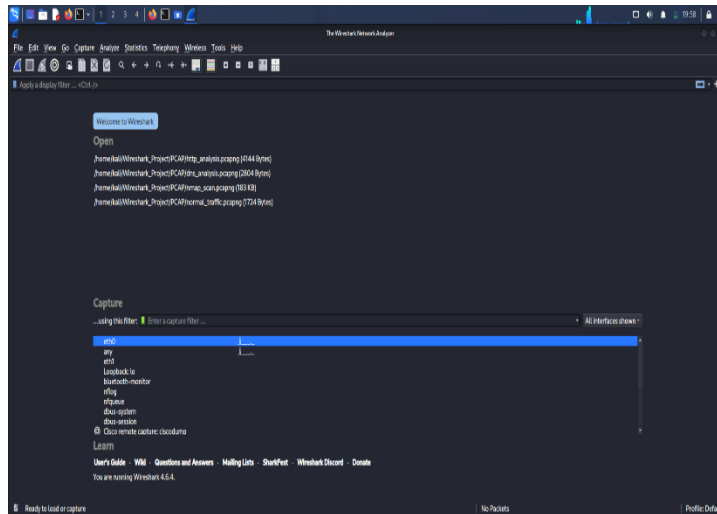


```
Session Actions Edit View Help
--(kali@kali)~$
--(kali@kali)~$ ifconfig
1: eth0: <BROADCAST,UP,LINK-UP,STATE-UP> 192.168.56.100 netmask 255.255.255.0 group default mtu 1500
    Link encap: Ethernet 08:00:27:00:12:34
    inet addr: 192.168.56.100 Bcast: 192.168.56.255 Mask: 255.255.255.0
    inet6 addr: fe80::208:1234:5678:9abc:1%eth0 prefixlen 64 scope link
    valid_lft forever preferred_lft forever
    Link encap: Ethernet 08:00:27:00:12:34
    inet6 addr: fe80::208:1234:5678:9abc:2%eth0 prefixlen 64 scope link
    valid_lft forever preferred_lft forever
2: wlan0: <BROADCAST,UP,LINK-UP,STATE-UP> 192.168.56.101 netmask 255.255.255.0 group default mtu 1500
    Link encap: Ethernet 08:00:27:00:12:35
    inet addr: 192.168.56.101 Bcast: 192.168.56.255 Mask: 255.255.255.0
    inet6 addr: fe80::208:1234:5678:9abc:3%wlan0 prefixlen 64 scope link
    valid_lft forever preferred_lft forever
3: wlan1: <BROADCAST,UP,LINK-UP,STATE-UP> 192.168.56.102 netmask 255.255.255.0 group default mtu 1500
    Link encap: Ethernet 08:00:27:00:12:36
    inet addr: 192.168.56.102 Bcast: 192.168.56.255 Mask: 255.255.255.0
    inet6 addr: fe80::208:1234:5678:9abc:4%wlan1 prefixlen 64 scope link
    valid_lft forever preferred_lft forever
--(kali@kali)~$ ping -c 1 192.168.56.101
PING 192.168.56.101 (192.168.56.101) 56(84) bytes of data:
64 bytes from 192.168.56.101: icmp_seq=1 ttl=64 time=0.012 ms
64 bytes from 192.168.56.101: icmp_seq=2 ttl=64 time=0.011 ms
64 bytes from 192.168.56.101: icmp_seq=3 ttl=64 time=0.010 ms
64 bytes from 192.168.56.101: icmp_seq=4 ttl=64 time=0.011 ms
--(kali@kali)~$ ping -c 1 192.168.56.102
PING 192.168.56.102 (192.168.56.102) 56(84) bytes of data:
64 bytes from 192.168.56.102: icmp_seq=1 ttl=64 time=0.013 ms
64 bytes from 192.168.56.102: icmp_seq=2 ttl=64 time=0.012 ms
64 bytes from 192.168.56.102: icmp_seq=3 ttl=64 time=0.011 ms
64 bytes from 192.168.56.102: icmp_seq=4 ttl=64 time=0.012 ms
--(kali@kali)~$
```

- Connectivity was tested using: ping 192.168.56.xxx
- Successful replies confirmed that the two machines could communicate.

1.7 WIRESHARK INSTALLATION AND SETUP

- Wireshark was installed on the kali Linux system itself.
- After launching Wireshark, the available network interfaces were checked.



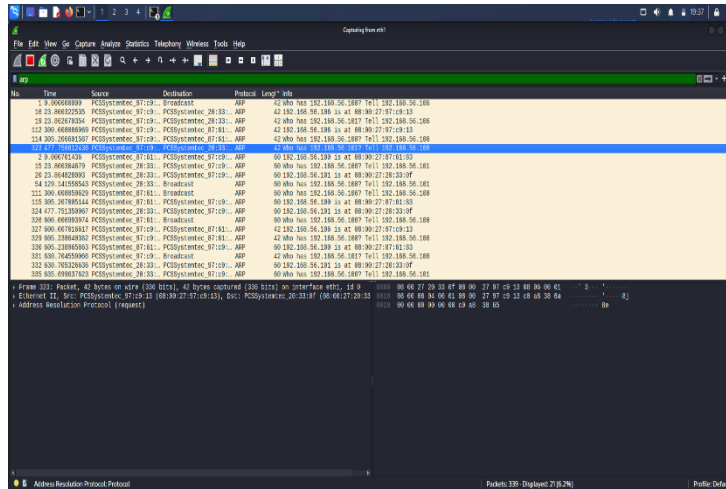
- The appropriate interface was selected and packet capture was started.
- Basic Process: -

1. Open Wireshark.
2. Identify the active network interface.
3. Double-click the interface.
4. Start packet capture.
5. Generate network traffic from kali.
6. Stop the capture.
7. Apply display filters.
8. Analyze packets.

1.8 ARP TRAFFIC ANALYSIS

- ARP stands for Address Resolution Protocol.
- It is used to determine the MAC address associated with an IP address within a local network.

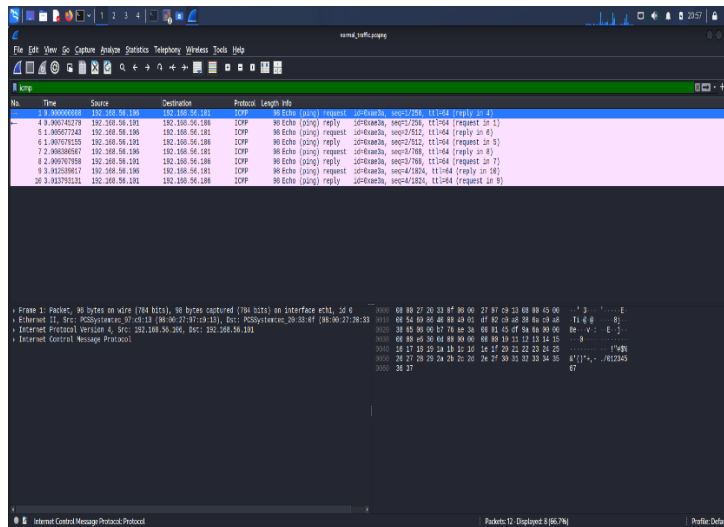
- To generate ARP traffic, communication between Kali and Metasploitable was initiated.
- Wireshark Filter: - arp



- The captured traffic showed ARP request and response packets.
- *ARP request:* -
The source machine asks: Who has this IP address?
- *ARP reply:* -
The destination responds with its MAC address.
- *Analysis:* -
ARP analysis helped understand how devices discover each other on a local network before communicating.

1.9 ICMP TRAFFIC ANALYSIS

- ICMP traffic was generated using the *ping* command.
ping 192.168.56.xxx
- Wireshark filter: - icmp



- The captured packets included: -

Echo Request

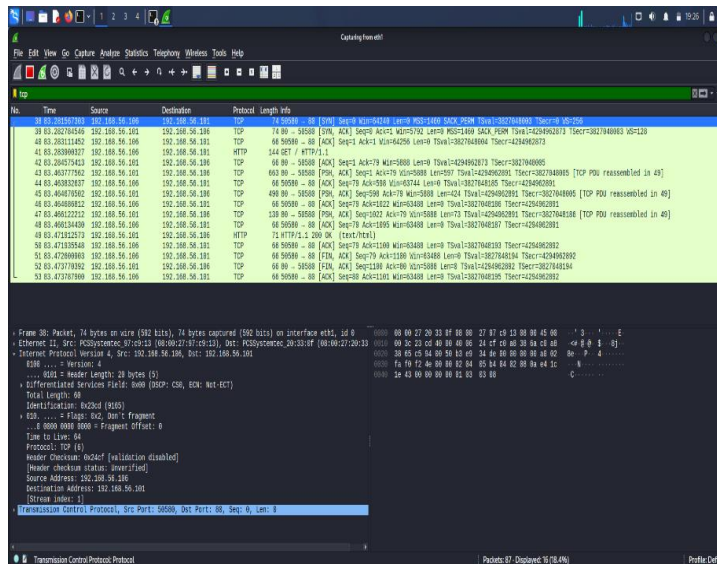
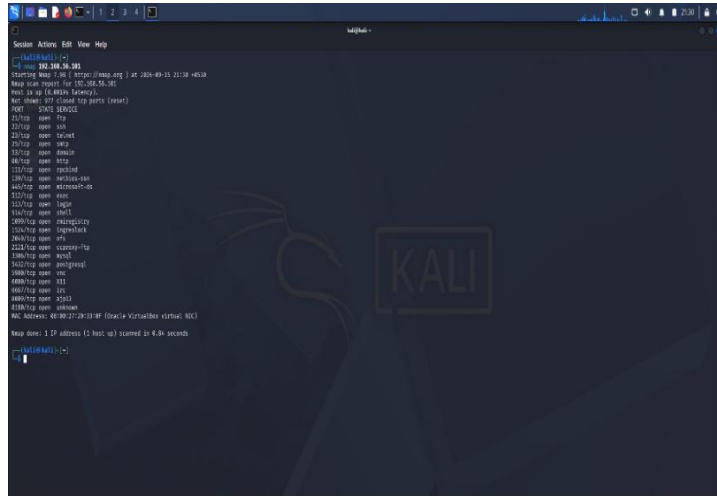
Echo Reply

- Analysis: -

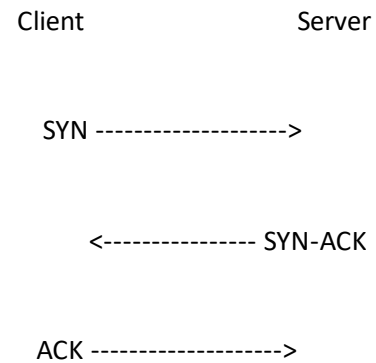
The Echo Request is sent by the source machine, while the Echo Reply is returned by the destination. This confirmed successful network communication.

1.10 TCP TRAFFIC ANALYSIS

- TCP is a connection-oriented protocol.
- TCP communication was analysed using Wireshark.
- Wireshark Filter: - tcp



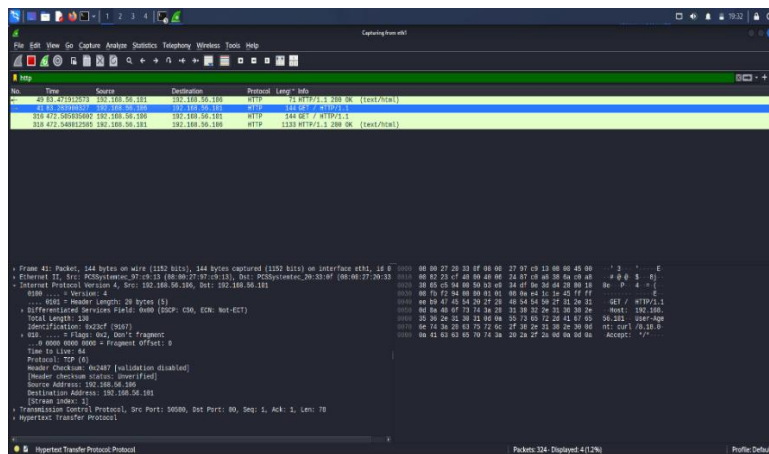
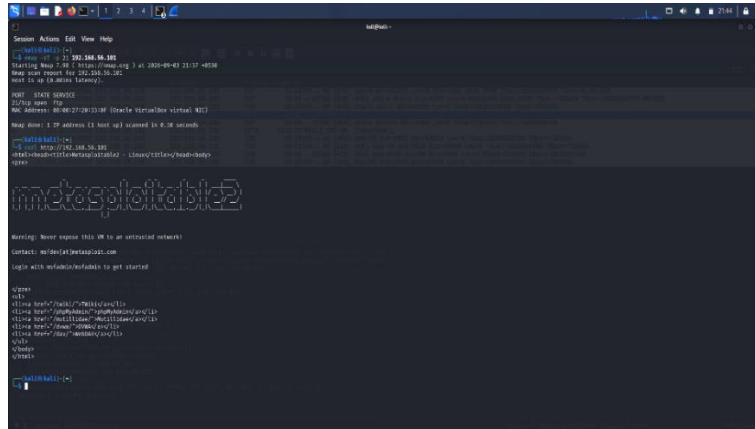
- Three-Way Handshake →



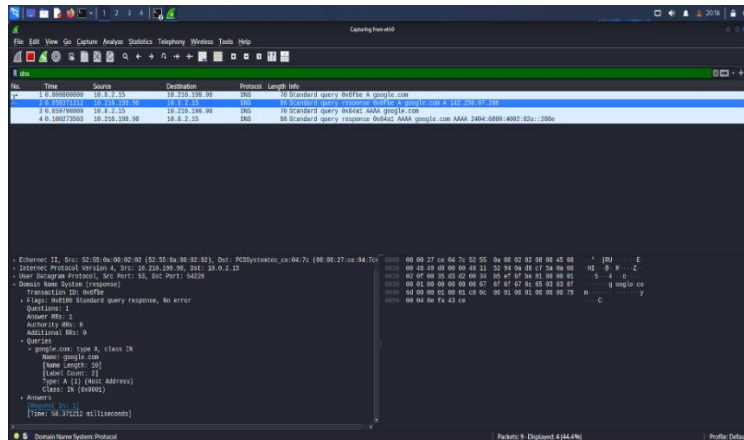
- Packet Analysis (TCP connection) →
 1. SYN – The client requests a TCP connection.
 2. SYN-ACK – The server acknowledges the request and responds.
 3. ACK – The client acknowledges the server's response.

1.11 HTTP TRAFFIC ANALYSIS

- HTTP traffic was generated in the lab environment.
- Wireshark filter: - http



- HTTP packets were examined to understand:
 - Source IP
 - Destination IP
 - Source port
 - Destination port



- The analysis included:
 - DNS queries
 - DNS responses
 - Query names
 - Source IP
 - Destination IP
 - DNS response information
- Example:

Client → DNS Server

DNS Query

DNS Server → Client

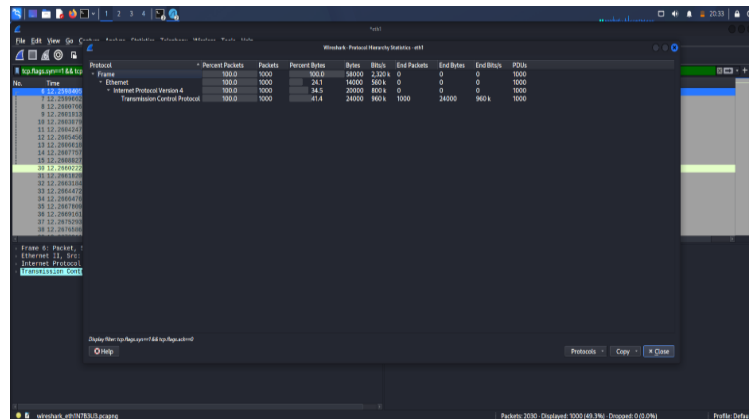
DNS Response

1.13 Nmap TRAFFIC ANALYSIS

- Nmap was used to generate scanning traffic from Kali.
- Example: `nmap 192.168.56.xxx`
- The resulting traffic was captured in Wireshark.
- Wireshark filter: - `tcp`
- The captured packets helped identify scanning behaviour, including TCP connection attempts to different ports.
- The traffic demonstrated how network scanning activity can appear in packet captures.

1.14 PROTOCOL HIERARCHY ANALYSIS

- Wireshark's Protocol Hierarchy feature was used to understand the protocols present in the captured traffic.
- Path:
Statistics -> Protocol Hierarchy

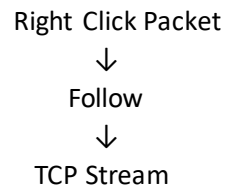


- It provided information about protocols such as:
 - Ethernet
 - ARP

- Number of packets
 - Number of bytes
 - Duration
- This helped identify which systems were communicating and how much traffic was exchanged.

1.16 TCP STREAM ANALYSIS

- TCP streams were analysed using:



- This feature reconstructs the communication belonging to a TCP connection.
- It helped visualize the complete communication between the client and server.

1.17 IMPORTANT WIRESHARK FILTERS USED

Filter	What it was used for
tcp.flags.syn == 1	Finding SYN packets during TCP/Nmap analysis
tcp.flags.ack == 1	Finding ACK packets
ip. addr == 192.168.56.xxx	Filtering traffic involving Metaspolitable
tcp. port == 80	HTTP/TCP port 80 traffic
tcp. port == 443	Displays TCP traffic involving port 443, associated with HTTPs

1.18 CONCLUSION

- This project provided practical experience in network traffic capture and packet analysis using Wireshark in a controlled cybersecurity environment. By analysing ARP, ICMP, TCP, HTTP, and DNS traffic, we developed an understanding of how different network protocols operate and how communication between systems can be observed at the packet level.
- Overall, this project strengthened our fundamentals in networking, packet analysis, troubleshooting, and cybersecurity investigation, and demonstrated how Wireshark can be used as an important tool for understanding and investigating network activity.